Emotion Decoder: How CNN Reads Faces and Feelings

Liran Ma, Zhipeng Cai

Introduction

It's a Monday morning. You wake up, look in the mirror, and give yourself a tiny smile. You know that smile means something — maybe you're feeling hopeful, maybe you're just trying to convince yourself to get through the day.

But now imagine this: You take a photo of that smile and feed it into a machine. Can it understand how you're feeling? Can it tell you're smiling not just with your lips, but with your heart?

This isn't just a poetic question — it's the challenge you'll explore in this hands-on project. You'll build a machine learning model based on a **Convolutional Neural Network (CNN)** that tries to answer a human question: "How are you feeling?"

You'll be in charge of tuning its vision — how it looks, what it remembers, and how it makes decisions — to help it understand emotions from images. Along the way, you'll learn how machines process visual information step by step.

Figure 1: Facing your own reflection — can a machine really understand the emotion behind your smile? *Image source:* rawpixel.com.

1 Why Should Machines Understand Emotion?

Emotion recognition isn't just a theoretical idea — it has practical uses in real life.

- **Customer Service:** Detecting user frustration or confusion can help chatbots respond more empathetically.
- **Driver Monitoring:** Monitoring drivers for fatigue or anger can reduce road accidents.
- **Assistive Technology:** Tools that help individuals with autism interpret facial expressions.

- **Mental Health Monitoring:** Detecting emotional cues in photos or videos could support early warning systems for depression.

Ethical Note: Just because we *can* estimate emotion doesn't mean we always *should*. Context, privacy, consent, and cultural nuance matter.

2 Why Use CNNs for Emotion Recognition?

Emotion recognition is a computer vision task: we are asking a machine to interpret visual information (faces, posture, scene). **Convolutional Neural Networks (CNNs)** are especially powerful here.

- **Local Feature Extraction:** Small filters scan the image to detect edges, textures, and local patterns (like eyebrows raising).
- **Hierarchical Understanding:** Shallow layers find simple edges; deeper layers assemble them into shapes (eyes, mouth), and finally into semantic concepts ("smile").
- **Data-Driven Learning:** CNNs learn which features matter; no need to hand-design edge detectors manually.
- **Proven Architecture:** They underpin modern systems in face analysis, medical imaging, object detection, and more.

Analogy: Think of each CNN layer as a team of specialists. Early specialists only notice basic strokes (like simple pencil lines). Later specialists combine these strokes into letters, then words, then whole sentences of meaning ("happy face").

3 How Machines See: Core Concepts

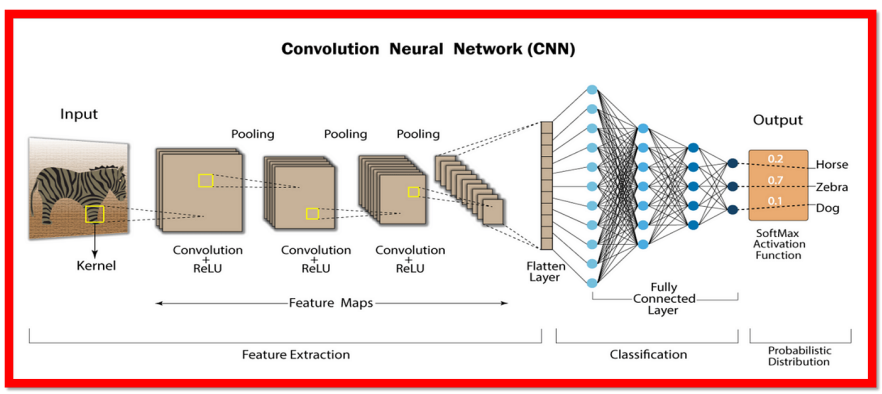

Figure 2: Overview of a Convolutional Neural Network (CNN) architecture, illustrating how different layers are connected. Source: Medium: Tech AI Made Easy

3.1 Convolution Basics

Idea: A convolution layer looks for local patterns. It slides a short *kernel* across the input, multiplies each covered value by a weight, adds them up to get a *score*, and then passes that score through an activation function (e.g., ReLU). A higher score means "this local piece matches the pattern this kernel is looking for."

1D Example (dice rolls) Imagine you record a die roll once per day. Take a small window that covers, say, 3 days. At each position:

- **Slide:** move the 3-day window one day at a time;
- **Multiply:** day 1 × weight 1, day 2 × weight 2, day 3 × weight 3;
- **Sum:** add those three products to get a *score* for this spot;
- **Activate:** apply a nonlinearity (e.g., ReLU) so the network can combine patterns later.

If the kernel puts bigger weight on "recent" days, the score becomes large exactly when the recent rolls are high. Deep-learning libraries usually implement this as *cross-correlation* (no kernel flip). For intuition, you can treat it as the same "slide–multiply–sum" step.

With Weights The kernel's weights are *learnable*. Training starts with rough values and then adjusts them so each kernel becomes a specialist:

- an "averaging" kernel smooths noise;
- a "difference/edge" kernel reacts to sudden rises;
- a "recent-days" kernel focuses on the last day or two.

Different weight patterns ⇒ different detectors.

2D for Images For images, the kernel is a small patch (e.g., 3×3). It scans across the picture and produces a *feature map*: bright responses where the pattern is present, dark where it is not. Some kernels light up on edges or stripes; others react to corners or small dots. Multiple kernels in one layer give multiple feature maps. Stacking layers lets the model combine simple cues (edges, dots) into complex shapes (digits, faces, etc.).

Process

$$\boxed{\text{Slide} \;\rightarrow\; \text{Multiply by weights} \;\rightarrow\; \text{Sum (score)} \;\rightarrow\; \text{Activation}}$$

Glossary. *stride* = step size when sliding; *padding* = add zeros around the border to control output size; *feature map* = the image/sequence of scores from one kernel.

3.2 Convolutional Layers (Feature Detectors)

A **convolutional layer** applies learnable small matrices called **kernels** (or **filters**) across the image. Each filter slides ("convolves") over the image, computing a weighted sum. The output forms a **feature map** highlighting where that pattern appears.

Key Idea: Different filters learn to detect different visual cues (edges, corners, curves, textures).

3.3 Kernel (Filter) Size

- **Small (3×3):** Focused on fine, local detail.
- **Large (7×7 or 11×11):** Broader context, but more parameters (risk of overfitting early).

Zoom Lens Analogy: Small kernels = magnifying glass on freckles; large kernels = stepping back to view the whole facial expression.

3.4 Number of Filters

Filters are like different pairs of glasses emphasizing different patterns. More filters = more patterns captured. But more filters also = more computation and overfitting risk if data is limited.

3.5 Pooling Layers (Optional Dimension Reduction)

- **Max Pooling:** Keeps the strongest signal (like retaining the boldest pencil mark).
- **Average Pooling:** Takes an average (smooth summary).

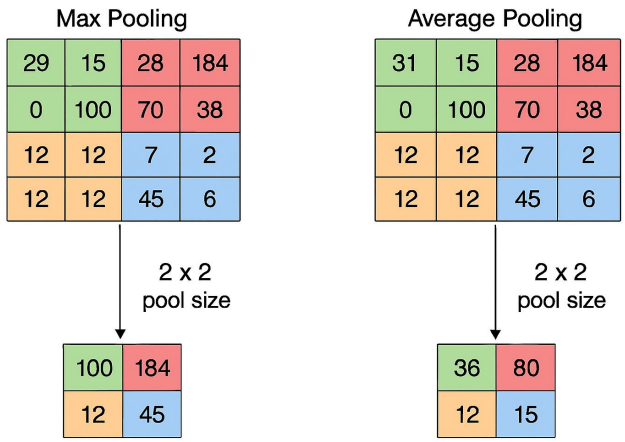

Figure 3: Illustration of 2×2 max pooling vs average pooling on a small patch. *Image source: ResearchGate.*

3.6 Fully Connected (Dense) Layers

After feature extraction, we *flatten* or globally pool the learned feature maps and feed them into **fully connected (FC)** layers. These integrate information across the entire face to make classification decisions (e.g., probabilities for "happy", "sad", etc.).

3.7 Comparison Summary

Convolutional Layers	Fully Connected Layers
Look locally (patch by patch)	Combine all features globally
Extract reusable patterns	Integrate patterns into a decision
Parameter sharing (efficient)	Many parameters (dense connections)
Spatial structure preserved	Spatial layout mostly discarded

3.8 Activation Functions in Neural Networks

Neural networks at a glance (Input–Hidden–Output)

- **Input layer**: takes raw features (e.g., pixels, words, numbers). No learned parameters here beyond optional normalization.

- **Hidden layers**: each hidden layer does a *linear step* (weighted sum + bias) followed by a *nonlinear activation*. These layers transform inputs into useful intermediate features.
- **Output layer**: converts the final hidden representation into a task-specific prediction. Activation depends on the task (e.g., sigmoid for binary probability, softmax for multi-class, identity for regression).

Figure 4: Pipeline diagram: Image → Hidden Layers → Output Image.

Why parameters? What do we tune?

- **Model parameters (learned from data)**: weights and biases in each layer. Training adjusts them to minimize a loss (e.g., cross-entropy, MSE).
- **Hyperparameters (chosen by you)**: learning rate, number of layers/units, activation type, regularization (weight decay, dropout), batch size, optimizer, etc. Tune these using a validation set or cross-validation.
- **Objective matters**: the loss defines what "good" means; different tasks imply different losses and sometimes different output activations.

How are weights chosen (and why activations matter)?

- **Initialization**: start from random small weights; good initialization helps gradients flow.
- **Optimization**: backpropagation + an optimizer (SGD/Adam) iteratively updates weights to reduce loss.
- **Role of activations**: activations shape output ranges and gradients. Proper choices prevent vanishing/exploding gradients and speed learning.

Activation output ranges ("from 0 to 1" and beyond) Different activations map z to different ranges, and the choice should match the task:

- **Sigmoid** maps to $(0, 1) \Rightarrow$ natural for probabilities (binary outputs).
- **Tanh** maps to $(-1, 1) \Rightarrow$ zero-centered hidden activations.
- **ReLU** maps to $[0, \infty) \Rightarrow$ fast and sparse, common in hidden layers.

Idea in one line. A neuron first makes a single score z from its inputs (a weighted sum plus a bias), then turns it into an output with an *activation function*:

$$a = g(z).$$

That small non-linear step $g(\cdot)$ is what lets deep networks model patterns that a simple line or plane cannot.

Why we need them

- Without non-linearity, stacking layers just behaves like *one* big linear layer.
- Activations shape the range of outputs (e.g., non-negative for ReLU; between 0 and 1 for sigmoid) and affect how gradients flow during training.

Common choices (simple intuition)

- **ReLU** $g(z) = \max(0, z)$: fast, works well in hidden layers, keeps positive signals and zeros out negatives. *Watch-out:* neurons can "die" if they only see negative inputs for a long time (fixes: LeakyReLU/ELU).
- **Sigmoid** $g(z) = \frac{1}{1+e^{-z}}$: outputs $(0, 1)$, great for *probabilities* (e.g., binary classification output). *Watch-out:* large $|z|$ makes gradients tiny (vanishing gradient).
- **Tanh** (outputs $(-1, 1)$): like a centered sigmoid; can work better than sigmoid in some hidden layers but can still saturate.

Practical tips

- Use **ReLU (or a ReLU variant)** in most hidden layers by default.
- Use **Sigmoid** only for outputs that represent probabilities in binary tasks.
- Use **Tanh** if you want zero-centered hidden activations and the network is shallow; otherwise prefer ReLU-family.

In training During backpropagation the network tweaks its weights to reduce loss. Good activation choices help gradients move cleanly through the layers (faster, more stable learning). Poor choices can slow learning or stall it.

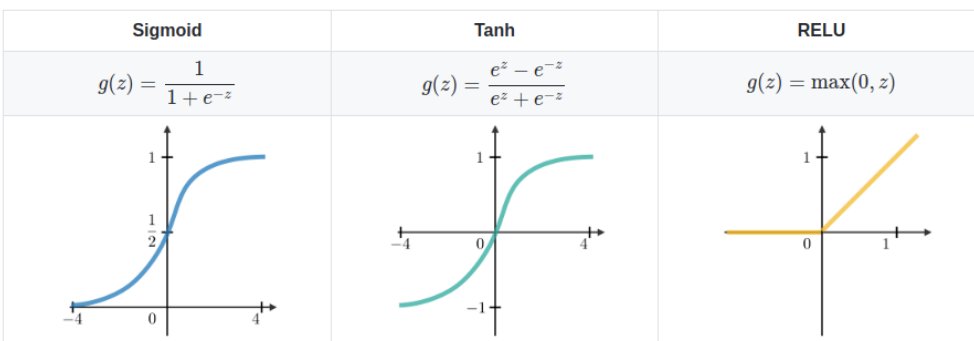

Figure 5: Shapes of common activations (Sigmoid, Tanh, ReLU) and where they saturate.

3.9 Dropout (Regularization)

Dropout randomly "turns off" a fraction (e.g., 30%) of connections during training. This prevents the network from becoming overly dependent on specific neurons.

> **Team Analogy:** If one star player (neuron) is benched randomly during practice, the rest of the team must learn to play well too.

4 Can You Read a Face? Exploring the EMOTIC Dataset

4.1 Human First

Before asking a machine, try yourself. Observe the face, posture, context (background objects, body language).

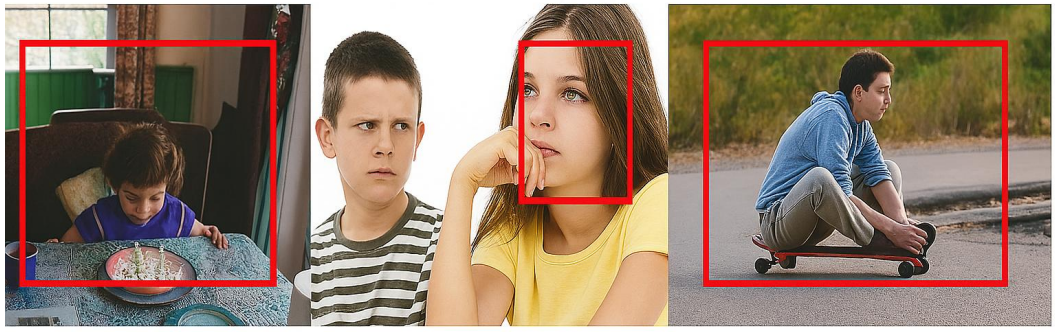

Figure 6: Example (placeholder) EMOTIC images. *Add proper dataset citation and licensing information (e.g., EMOTIC dataset paper).*

Try This: Write down your emotion guess for 5 images. Later, compare to the model's top probability predictions. Where did you disagree most? Why?

5 Interactive Exploration

5.1 Set Up the Model

Parameters you can vary:

- Number of convolutional layers
- Filter count and kernel size
- Pooling strategy (or none)
- Dropout rate(s)
- Number / size of fully connected layers
- Optimizer choice (SGD, Adam, etc.)
- Learning rate (LR)
- Loss function
- Number of epochs + early stopping

5.2 Baseline CNN

Start minimal:

- Conv: 32 filters, 3×3
- Dense: 128 units
- Optimizer: Adam
- Loss: Categorical cross-entropy
- Epochs: e.g. 10–20

Experiment Protocol: Change *one* hyperparameter at a time. Record: (1) change, (2) hypothesis, (3) result, (4) reflection.

Adjustment	Change	Reason	Result (Val Acc)	Reflection
Kernel size	3x3 → 5x5	Capture broader context early	0.68 → 0.72	Helped; more context beneficial
Filters	32 → 64	Capture more feature types	0.72 → 0.75	Gains tapering; watch overfitting
Dropout	None → 0.3	Reduce overfitting	0.75 → 0.74	Slight dip; perhaps underfitting

Table 1: CNN Tuning Journal (Example Entries)

5.3 Compare Multiple Models

Model ID	Conv Layers	Filters (1st)	Dropout	Optimizer	Validation Accuracy
A	1	32	0.0	SGD	
B	2	32	0.5	Adam	
C	3	64	0.3	SGD	
D	3	64	0.3	Adam	
E	4	64	0.4	Adam	

Reflection Questions

- Which model balanced accuracy and training time best?
- Did deeper always mean better?
- Did dropout help at the moment overfitting began?
- How stable were your results across multiple runs (random seeds)?

Key Lesson: Larger models can memorize; well-regularized moderate models often generalize better.

5.4 Comprehensive Experiment Log

Conv Layers	Filters	Kernel	Dropout	Optimizer	FC Layers	FC Units	LR	Val. Acc.
1	32	3	0.1	SGD	1	128	0.001	0.68
3	64	5	0.3	Adam	2	256	0.0005	0.75
2	64	3	0.2	Adam	1	256	0.001	0.73
4	64	3	0.4	Adam	2	512	0.0005	0.74
2	32	5	0.0	SGD	1	128	0.002	0.66
...	...	...	...	...	...	...	...	...

6 Optimizers (Deep Dive)

An **optimizer** updates the model's parameters (weights) to reduce the loss.

6.1 Stochastic Gradient Descent (SGD)

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

- θ_t: current weights.
- η: learning rate.
- $\nabla_\theta \mathcal{L}$: gradient (direction of steepest increase in loss); we move opposite it.

Plain Language: Look at how wrong we are (loss), compute direction to get less wrong (gradient), take a small step that way.

6.2 Momentum (Improved SGD)

Adds a velocity term to smooth updates:

$$v_{t+1} = \gamma v_t + \eta \nabla_\theta \mathcal{L}(\theta_t), \quad \theta_{t+1} = \theta_t - v_{t+1}$$

Helps accelerate in consistent directions; reduces zig-zag.

6.3 Adam (Adaptive Moment Estimation)

Adam keeps track of *moving averages* of gradients and squared gradients.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

Bias-correct:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

Update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Analogy: Adam is like a runner who not only remembers the average direction of the hills (first moment) but also how rough the terrain is (second moment) and adjusts step size automatically.

6.4 When to Pick Which?

Optimizer	Pros	Cons
SGD	Simple; good generalization; stable	Slower convergence; needs LR tuning
Momentum SGD	Faster than plain SGD; smooth	Still LR sensitive
Adam	Fast convergence; less LR tuning	Sometimes overfits; may plateau sub-optimally
AdamW	Better regularization via decoupled weight decay	Slightly more setup complexity

6.5 Intuitive Example

Imagine climbing down a bumpy hill to reach the lowest valley:

- SGD: One step at a time using immediate slope.
- Momentum: Remembers previous direction, glides past small bumps.
- Adam: Adjusts step size individually for each dimension (direction).

7 Learning Rate (LR)

7.1 What Is It?

The learning rate η controls how big each weight update step is.

Too High: Model bounces and never settles (diverges).
Too Low: Model inches slowly; may get stuck in a shallow dip.

7.2 Local vs Global Minima

Local minimum: A dip that is lower than neighbors but not the lowest overall. **Global minimum:** The deepest valley (lowest possible loss).

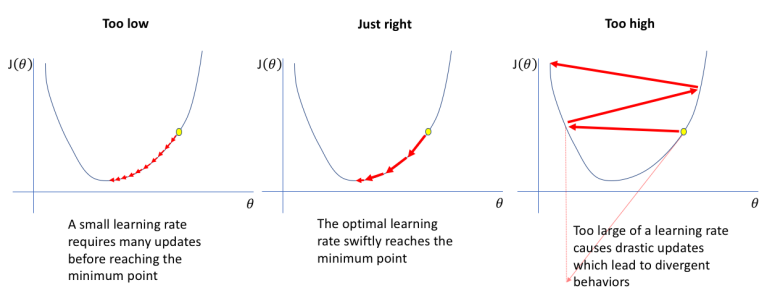

Figure 7: Effect of different learning rates on neural network training convergence. Source: jeremyjordan.me

7.3 Learning Rate Schedules

- **Step Decay:** Reduce LR every k epochs.
- **Exponential Decay:** $\eta_t = \eta_0 \cdot \gamma^t$.
- **Warm Restarts (Cosine):** Periodically raise LR to escape local minima.

- **Cyclical LR:** Oscillates between bounds; can improve exploration.

7.4 Practical Tip: LR Finder

Run a mini training where LR increases gradually; plot loss vs LR; choose LR before loss explodes.

8 Loss Function: Categorical Cross-Entropy

8.1 Plain Definition

Categorical cross-entropy measures how far the model's predicted probability distribution is from the true distribution (which puts all probability on the correct class).

8.2 Mathematical Formula

For a single example with C classes, true one-hot vector $\mathbf{y}$ and predicted probabilities $\hat{\mathbf{y}}$:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$$

Because only the correct class k has $y_k = 1$, this simplifies to:

$$\mathcal{L} = -\log(\hat{y}_k)$$

Interpretation: If the model assigns high probability to the correct class, $\hat{y}_k$ is near 1 and $-\log(\hat{y}_k)$ is near 0 (good). If it is unsure (e.g., 0.2), the loss is larger.

8.3 Confidence and Wrong Answers

If the model is confidently wrong (predicts 0.99 for the wrong class), the loss is huge. This punishes overconfident mistakes.

8.4 Relation to Information Theory

Cross-entropy is the expected number of bits needed to encode the true labels using the model's predicted distribution.

9 Accuracy vs Loss

9.1 Why Both Metrics Matter

- **Accuracy:** Percentage of correct top predictions.
- **Loss:** Measures *quality* and *confidence* of predictions.

9.2 High Accuracy, High Loss Scenario

Suppose 10 classes and the model is correct 80% of time, but assigns only 0.30 probability to the correct class each time. Accuracy appears good, yet loss remains high (model uncertain).

9.3 Low Accuracy, Lower Loss Scenario

Early in training, a model might misclassify often but produce moderately spread probabilities (less overconfident), giving a lower loss trend that indicates learning.

Try This: Track both validation accuracy and validation loss. Note any epochs where accuracy plateaus but loss still decreases — interpret what that means about calibration.

10 Epochs and Early Stopping

10.1 Epoch Definition

One epoch = one complete pass through all training examples.

Analogy: Studying a chapter once (1 epoch). Re-reading the chapter 20 times (20 epochs) can help retention, but after a point you might memorize sentences (overfitting) rather than concepts.

10.2 Underfitting vs Overfitting Curves

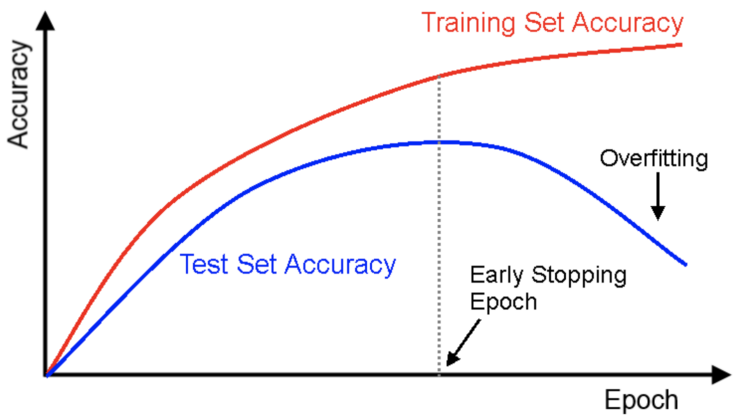

Figure 8: Illustration of overfitting in neural network training, showing how validation error starts increasing after a certain point. Source: Data Science Stack Exchange

10.3 Early Stopping

Monitor validation loss. If it does not improve for, say, 5 consecutive epochs (a **patience** value), stop training and revert to best weights.

Warning: Training longer is not always better. More epochs can degrade performance on new data.

Try This: Train once with early stopping, once without. Compare confusion matrices.

11 Special Cases: More Is Not Always Better

11.1 Overfitting vs Underfitting Summary

Overfitting	**Underfitting**
Memorizes noise/patterns	Fails to learn core patterns
High train acc, lower val acc	Both train and val acc low
Large gap train–val	Small gap but poor scores
Solution: regularize, simplify	Solution: increase capacity, train more

11.2 Bias–Variance Intuition

- **High Bias (Underfit):** Model assumptions too rigid.
- **High Variance (Overfit):** Model sensitive to training fluctuations.

11.3 Regularization Tools

- Dropout
- Data augmentation (flips, crops, brightness)
- Weight decay (L2)
- Early stopping
- Simpler architecture

12 Putting It Into Practice (Workflow)

1. **Inspect Data:** Class balance? Image resolution? Ethical constraints?
2. **Split Data:** Train / Validation / (Optional) Test.
3. **Baseline Model:** Minimal CNN to set a reference.
4. **Iterative Tuning:** Change one hyperparameter at a time.
5. **Track Metrics:** Log training and validation loss, accuracy, maybe F1.
6. **Stop Smart:** Use early stopping or LR schedule.
7. **Evaluate:** Compare models with consistent metrics.
8. **Reflect:** Why did certain changes help or hurt?

Checkpoint Question **Q.** What happens if we train a CNN on a different dataset whose labels are random (i.e., inaccurate or unrelated to the images)?

A. The model will not learn a meaningful mapping. Because the labels carry no real signal about the inputs, the network's predictions stay near chance level. Typical signs:

- Training and validation accuracy hover around chance (e.g., roughly 1/num_classes); the loss plateaus.
- A very large network trained long enough may *memorize* the random training labels (high training accuracy), but validation accuracy remains at chance—classic overfitting with no generalization.
- The confusion matrix is nearly uniform; predictions are unstable and inconsistent.

Takeaway. When labels are random, a CNN cannot learn useful features. Data quality—not just model size or training time—determines whether learning is possible.

13 Ethical Reflection: Can a Machine Really "Feel"?

Machines map patterns → labels. They do *not* experience feelings. Misclassifications can have human consequences (e.g., misjudging a person's emotional state).

Questions

- Did the model misinterpret culturally specific expressions?
- How might lighting, occlusions, or masks affect fairness?
- How will you communicate uncertainty (probabilities, confidence intervals)?

Bonus Challenge: Carefully test the model on your own photo (if privacy and consent conditions are respected). Did it generalize?

Appendix A: Example Code

Listing 1: Example CNN Training

```
# Build the model
model = build_model(
    conv_layers.value,
    conv_filters.value,
    kernel_size.value,
    fc_layers.value,
    fc_units.value,
    dropout_rate.value
)

# Use a constant learning rate
if OPTIM == 'adam':
    opt = optimizers.Adam(learning_rate=BASE_LR)
else:
    opt = optimizers.SGD(learning_rate=BASE_LR, momentum=0.9)

model.compile(optimizer=opt,
              loss=LOSS,
              metrics=['accuracy'])

print("Model Summary:")
model.summary()

# Early stopping
es = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

# Train
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=[es]
```

```
)

# Plot accuracy
plt.figure()
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='val')
plt.title('Accuracy')
plt.legend()
plt.show()

# Plot loss
plt.figure()
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='val')
plt.title('Loss')
plt.legend()
plt.show()

# Final evaluation
loss_val, acc_val = model.evaluate(test_ds, verbose=0)
print(f"\nTest Loss: {loss_val:.4f}   Test Accuracy: {acc_val:.4f}")
```

Appendix B: Glossary (Beginner Friendly)

Parameter
A number the model learns (weights, biases).

Filter / Kernel
Small matrix scanning the image to detect a pattern.

Feature Map
Output of applying a filter.

Activation Function
Adds non-linearity (e.g., ReLU).

Learning Rate
Controls update step size.

Epoch
One full pass over training data.

Regularization
Techniques reducing overfitting.

Softmax
Converts raw scores to probabilities that sum to 1.

Cross-Entropy
Measures mismatch between predicted and true distributions.

References and Image Attributions

References

[1] *EMOTIC Dataset.*

[2] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[4] D. P. Kingma, J. Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.

[5] N. Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 2014.

[6] Prechelt, L. Early Stopping — But When? In: *Neural Networks: Tricks of the Trade*, 1998.

[7] Loshchilov, Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. *ICLR*, 2017.

[8] Smith. Cyclical Learning Rates for Training Neural Networks. *WACV*, 2017.

"Machines classify patterns; humans feel meaning."